14_TECH_STACK.md

UNPLND Technology Stack

Version 1.0

Status: Approved

Purpose

This document defines the official technology stack for UNPLND.

Every implementation must follow these decisions.

Do not replace technologies without updating this document.

Philosophy

Choose mature technologies.

Choose scalable technologies.

Choose technologies with large communities.

Avoid experimental dependencies.

Long-term maintainability always wins.

Platform

Primary

Flutter

Reason

Single codebase

Native performance

Excellent UI capabilities

Fast iteration

Cross-platform support

Programming Language

Dart

Latest stable version.

Use modern language features.

Enable strict analysis.

No deprecated APIs.

Architecture

Architecture Style

Feature-first

Layered

Modular

Scalable

Structure

presentation/

domain/

data/

core/

shared/

features/

Never mix responsibilities.

State Management

Riverpod

Reason

Compile-time safety

Excellent testability

Scalable architecture

Predictable state

Avoid global mutable state.

Routing

GoRouter

Support

Deep Links

Authentication Guards

Nested Navigation

Future Web Support

Navigation must remain declarative.

Backend

Firebase

Services

Authentication

Firestore

Storage

Cloud Functions

Messaging

Crashlytics

Analytics

Remote Config (future)

App Check

Authentication

Supported

Apple Sign In

Google Sign In

Email + Password

Anonymous Session (optional)

Future

Passkeys

Phone Authentication

Database

Firestore

Collections must remain normalized.

Avoid deeply nested structures.

Prefer references.

File Storage

Firebase Storage

Stores

Profile Images

Album Images

Temporary Uploads

Generated Covers

Maps

Google Maps SDK

Required APIs

Maps

Places

Directions

Geocoding

Autocomplete

Future

Routes API

AI Provider

Use an abstraction layer.

Never couple the application to one provider.

Supported

OpenAI-compatible APIs

Anthropic-compatible APIs

Gemini-compatible APIs

Future providers

The application communicates only with the AI service layer.

Never directly with providers from UI code.

Weather

Dedicated Weather Provider

Cached

15 minutes

Fallback

Last successful response

Notifications

Firebase Cloud Messaging

Local Notifications

Deep Linking

Notification Categories

Reminder Notifications

Album Ready

Plan Reminder

Weather Changes

Analytics

Firebase Analytics

Only meaningful events.

No excessive tracking.

Privacy first.

Crash Reporting

Firebase Crashlytics

Every unexpected crash

↓

Automatically reported

No personal data inside logs.

Dependency Injection

Riverpod Providers

No Service Locator.

No Singleton abuse.

Networking

HTTP Client

Dio

Requirements

Interceptors

Retry

Timeout

Logging

Authentication

Caching

Local Storage

Hive

Stores

Theme

Settings

Cached Plans

Favorites

Recent Searches

Never store sensitive tokens insecurely.

Secure Storage

Flutter Secure Storage

Stores

Access Tokens

Refresh Tokens

Sensitive Keys

Image Handling

Cached Network Image

Compression

Lazy Loading

Progressive Loading

Memory Optimization

Forms

Flutter Form

Validation

Reusable Validators

Input Formatters

Live Validation

Testing

Unit Tests

Widget Tests

Integration Tests

Golden Tests

Target Coverage

80%+

Business logic must always be tested.

CI/CD

GitHub

GitHub Actions

Automatic Formatting

Automatic Tests

Automatic Build

Automatic Release Tags

Linting

flutter_lints

custom analysis options

Strict mode

Warnings treated seriously.

Formatting

dart format

Entire project formatted automatically.

Never commit unformatted code.

Naming Convention

Classes

PascalCase

Variables

camelCase

Files

snake_case

Constants

camelCase unless language conventions require otherwise.

Consistency over preference.

Assets

assets/

images/

icons/

illustrations/

animations/

fonts/

Never place assets randomly.

Environment Variables

Separate

Development

Staging

Production

Never hardcode secrets.

Security

HTTPS only.

Certificate validation.

Firestore Security Rules.

Input validation.

Rate limiting.

Least privilege principle.

Performance Targets

Cold Start

<2 seconds

Frame Rate

60 FPS minimum

Target

120 FPS

Memory Usage

Optimized

Battery Usage

Minimal

Scalability

Architecture should support

1M+ users

Without major rewrites.

Documentation

Every public class

↓

Documented

Every complex method

↓

Explained

Every feature

↓

Own documentation

Golden Rule

Technology exists to support the product.

Never choose technology because it is trendy.

Choose what is reliable.

Claude Instructions

Never replace technologies.

Never introduce additional frameworks without approval.

Reuse existing architecture.

Follow the established folder structure.

Always think long-term.

Build software that can still be maintained in five years.

END OF 14_TECH_STACK.md